

Answerspot — Recommendation Engine (M5)

The core value engine. Turns scan results into ranked, plain-language fixes — each with a generated artifact where possible. This is what justifies the subscription; monitoring alone doesn't.

```
apps/workers/answerspot_workers/recommendations/  
├─ __init__.py  
├─ signals.py          # derive facts from scan_results (what's true)  
├─ rules.py           # rules that fire on signals (what to recommend)  
├─ artifacts.py       # generate the actual fix (schema, copy, keywords)  
├─ scoring.py         # visibility score + weights (ADR 0005)  
└─ engine.py          # orchestration: signals -> rules -> persist
```

Design principles baked in:

- **Plain language, no jargon.** "citations" → "which sites the AI trusts."
- **Hand them the fix, not homework.** Every rule that can generate an artifact does.
- **Ranked by impact, deduped, capped.** Operators see the top few, not 30.
- **Deterministic core, LLM at the edges.** Rules are testable code; the LLM only drafts human-facing copy (review responses), never decides what fires.

`signals.py` — derive facts

Signals are plain facts computed from a business's recent `scan_results`. Rules read signals, never raw DB rows. This keeps rules pure and testable.

```
python
```

```

from dataclasses import dataclass
from collections import Counter

@dataclass
class QuerySignal:
    query_text: str
    mentioned: bool
    rank_position: int | None
    competitors: list[str]          # names appearing in this query's answers
    has_citations_for_you: bool
    sentiment: str                  # POSITIVE | NEUTRAL | NEGATIVE | UNKNOWN

@dataclass
class BusinessSignals:
    business_name: str
    category: str
    city: str
    queries: list[QuerySignal]

    # — Derived aggregates rules use —
    @property
    def appearance_rate(self) -> float:
        if not self.queries:
            return 0.0
        return sum(q.mentioned for q in self.queries) / len(self.queries)

    @property
    def missing_queries(self) -> list[QuerySignal]:
        return [q for q in self.queries if not q.mentioned]

    @property
    def negative_queries(self) -> list[QuerySignal]:
        return [q for q in self.queries if q.sentiment == "NEGATIVE"]

    @property
    def top_competitors(self) -> list[tuple[str, int]]:
        """Competitors ranked by how many of YOUR queries they appear in."""
        c = Counter()
        for q in self.queries:
            for name in q.competitors:
                if name != self.business_name:

```

```

        c[name] += 1
    return c.most_common()

def competitor_beats_you_in(self, competitor: str) -> list[QuerySignal]:
    """Queries where competitor appears and you don't."""
    return [q for q in self.queries if competitor in q.competitors and not q.mentioned]

@property
def citation_gap_queries(self) -> list[QuerySignal]:
    """Mentioned but with no sources backing you – weak, fragile mentions."""
    return [q for q in self.queries if q.mentioned and not q.has_citations_for_you]

```

rules.py — what to recommend

Each rule is a pure function: `(BusinessSignals -> Recommendation | None)`. Rules declare their type, severity, and an impact weight used for ranking. Adding a new recommendation type = adding one function to the registry.

python

```

from dataclasses import dataclass, field
from typing import Callable, Optional
from .signals import BusinessSignals
from . import artifacts

@dataclass
class DraftRecommendation:
    type: str
    severity: str          # LOW | MEDIUM | HIGH | CRITICAL
    title: str
    message: str
    impact: float          # 0..1, drives ranking
    artifact: Optional[dict] = None  # { kind, content } or None

# — Individual rules —

def rule_missing_high_intent(s: BusinessSignals) -> Optional[DraftRecommendation]:
    """You're absent from queries competitors are winning."""
    missing = s.missing_queries
    if not missing:
        return None
    # Prioritize the miss with the most competitors present (hottest slot).
    hottest = max(missing, key=lambda q: len(q.competitors))
    if not hottest.competitors:
        return None
    n = len(hottest.competitors)
    return DraftRecommendation(
        type="KEYWORD_GAP",
        severity="HIGH" if n >= 2 else "MEDIUM",
        title=f'Get listed for "{hottest.query_text}"',
        message=(
            f"You don't appear when customers search \"{hottest.query_text}\", "
            f"but {n} competitor{'s' if n > 1 else ''} do. "
            f"Add a clear section about this service to your site so AI assistants ca
        ),
        impact=0.6 + 0.1 * min(n, 3),
        artifact=artifacts.keyword_section(hottest.query_text, s.category, s.city),
    )

def rule_citation_gap(s: BusinessSignals) -> Optional[DraftRecommendation]:

```

```

"""Mentioned but nothing backs you up – add structured data / FAQ."""
gaps = s.citation_gap_queries
if len(gaps) < 2:  # only worth flagging if it's a pattern
    return None
return DraftRecommendation(
    type="CITATION_GAP",
    severity="HIGH",
    title="Add FAQ structured data to your website",
    message=(
        f"You're mentioned in {len(gaps)} queries but no sources back you up, "
        f"so those mentions are fragile. FAQ structured data helps AI assistants "
        f"quote you directly and trust your site."
    ),
    impact=0.7,
    artifact=artifacts.faq_schema(s.category, s.city),
)

```

```

def rule_review_response(s: BusinessSignals, response_rate: float, review_count: int)
    """Low review-response rate weakens a trust signal AI weights."""
    if review_count < 5 or response_rate >= 0.6:
        return None
    return DraftRecommendation(
        type="REVIEW_SIGNAL",
        severity="MEDIUM",
        title="Reply to your unanswered reviews",
        message=(
            f"You respond to about {round(response_rate * 100)}% of your {review_count} "
            f"AI assistants treat response rate as a sign of an engaged, trustworthy "
            f"Replying to a few more can lift how you're described."
        ),
        impact=0.45,
        artifact=artifacts.review_response_draft(s.business_name),
    )

```

```

def rule_negative_sentiment(s: BusinessSignals) -> Optional[DraftRecommendation]:
    """AI is describing you negatively somewhere – highest priority."""
    neg = s.negative_queries
    if not neg:
        return None
    return DraftRecommendation(
        type="SENTIMENT_ISSUE",
        severity="CRITICAL",

```

```

        title="AI is describing you negatively",
        message=(
            f"In {len(neg)} quer{'ies' if len(neg) > 1 else 'y'}, AI assistants descr
            f"business in a negative light. This is worth investigating first – it ca
            f"gains everywhere else."
        ),
        impact=0.95,
        artifact=None,    # needs human investigation, not a canned fix
    )

def rule_competitor_overtake(s: BusinessSignals) -> Optional[DraftRecommendation]:
    """A single rival dominates your queries."""
    top = s.top_competitors
    if not top:
        return None
    name, count = top[0]
    beats = s.competitor_beats_you_in(name)
    if len(beats) < 2:
        return None
    return DraftRecommendation(
        type="COMPETITOR_OVERTAKE",
        severity="HIGH",
        title=f"{name} is winning your slots",
        message=(
            f"{name} appears in {count} of your queries and beats you in {len(beats)}
            f"Focus your next improvements on the services where they're pulling ahead
        ),
        impact=0.65,
        artifact=None,
    )

# — Registry —
# Rules needing extra inputs (reviews) are wrapped by the engine.
SIGNAL_ONLY_RULES: list[Callable[[BusinessSignals], Optional[DraftRecommendation]]] =
    rule_missing_high_intent,
    rule_citation_gap,
    rule_negative_sentiment,
    rule_competitor_overtake,
]

```

`artifacts.py` — generate the actual fix

The "hand them the fix" layer. Deterministic templates for structured output (schema, keyword lists); LLM only for natural-language drafts (review replies), and even then behind a clean interface so it's swappable/testable.

```
python
```

```

import json

def faq_schema(category: str, city: str) -> dict:
    """Copy-paste FAQ structured data seeded with the operator's context."""
    payload = {
        "@context": "https://schema.org",
        "@type": "FAQPage",
        "mainEntity": [
            {
                "@type": "Question",
                "name": f"Do you offer emergency {category.lower()} service?",
                "acceptedAnswer": {
                    "@type": "Answer",
                    "text": f"Yes – we offer fast, reliable {category.lower()} service",
                },
            },
            {
                "@type": "Question",
                "name": f"What areas do you serve?",
                "acceptedAnswer": {
                    "@type": "Answer",
                    "text": f"We serve {city} and the surrounding area.",
                },
            },
        ],
    }
    content = (
        '<script type="application/ld+json">\n'
        + json.dumps(payload, indent=2)
        + "\n</script>"
    )
    return {"kind": "code", "content": content}

def keyword_section(query_text: str, category: str, city: str) -> dict:
    """A ready-to-paste page section targeting a missed query."""
    heading = query_text.title()
    content = (
        f"## {heading}\n\n"
        f"Looking for {query_text}? Our {city}-based team specializes in exactly this"
        f"We provide professional {category.lower()} services with transparent pricin"
        f"fast response times. Contact us today for a free quote.\n"
    )

```



```

)
return {"kind": "text", "content": content}

def review_response_draft(business_name: str) -> dict:
    """LLM-drafted reply template. Falls back to a safe default if LLM is off."""
    draft = _llm_review_reply(business_name)
    return {"kind": "text", "content": draft}

def _llm_review_reply(business_name: str) -> str:
    # TODO(M5): call the LLM with a tight prompt + guardrails. Deterministic
    # fallback below keeps the engine working (and tests green) without a key.
    return (
        f"Thank you so much for the kind words! It was a pleasure helping you. "
        f"We really appreciate you taking the time to share your experience. "
        f"— The {business_name} team"
    )

```

scoring.py — visibility score (ADR 0005)

Deterministic. Weights live here (versioned), and every score stores the weights it used so historical trend lines stay interpretable when weights change.

python

```

from dataclasses import dataclass
from .signals import BusinessSignals

# ADR 0005 launch weights. Change = new ADR + note in UI. Never mutate silently.
WEIGHTS = {"w1": 0.40, "w2": 0.30, "w3": 0.15, "w4": 0.15}
WEIGHTS_VERSION = "2026-07-10"

@dataclass
class ScoreResult:
    score: int
    breakdown: dict

def _rank_score(signals: BusinessSignals) -> float:
    ranks = [q.rank_position for q in signals.queries if q.rank_position]
    if not ranks:
        return 0.0
    # rank 1 -> 1.0, rank 5+ -> ~0. Linear, clamped.
    return sum(max(0.0, (6 - r) / 5) for r in ranks) / len(ranks)

def _sentiment_score(signals: BusinessSignals) -> float:
    vals = {"POSITIVE": 1.0, "NEUTRAL": 0.5, "NEGATIVE": 0.0}
    scored = [vals[q.sentiment] for q in signals.queries if q.sentiment in vals]
    return sum(scored) / len(scored) if scored else 0.5

def _citation_score(signals: BusinessSignals) -> float:
    mentioned = [q for q in signals.queries if q.mentioned]
    if not mentioned:
        return 0.0
    return sum(q.has_citations_for_you for q in mentioned) / len(mentioned)

def compute_score(signals: BusinessSignals) -> ScoreResult:
    appearance = signals.appearance_rate
    rank = _rank_score(signals)
    sentiment = _sentiment_score(signals)
    citation = _citation_score(signals)

    raw = (
        WEIGHTS["w1"] * appearance

```

```
        + WEIGHTS["w2"] * rank
        + WEIGHTS["w3"] * sentiment
        + WEIGHTS["w4"] * citation
    )
    return ScoreResult(
        score=round(raw * 100),
        breakdown={
            "appearanceRate": round(appearance, 3),
            "rankScore": round(rank, 3),
            "sentimentScore": round(sentiment, 3),
            "citationScore": round(citation, 3),
            "weights": WEIGHTS,
            "weightsVersion": WEIGHTS_VERSION,
        },
    )
```

`engine.py` — orchestration

Loads signals, runs rules, ranks/dedupes/caps, computes the score, persists. This replaces the

`TODO(M5)` in `tasks/recommender.py`.

```
python
```

```

from .signals import BusinessSignals
from .rules import SIGNAL_ONLY_RULES, rule_review_response, DraftRecommendation
from .scoring import compute_score

MAX_RECOMMENDATIONS = 5 # operators see the top few, not everything

def generate(signals: BusinessSignals, *, review_rate: float, review_count: int) -> d
    drafts: list[DraftRecommendation] = []

    for rule in SIGNAL_ONLY_RULES:
        rec = rule(signals)
        if rec:
            drafts.append(rec)

    review_rec = rule_review_response(signals, review_rate, review_count)
    if review_rec:
        drafts.append(review_rec)

    # Dedupe by type (one per type), keep the highest-impact instance.
    best_by_type: dict[str, DraftRecommendation] = {}
    for d in drafts:
        cur = best_by_type.get(d.type)
        if cur is None or d.impact > cur.impact:
            best_by_type[d.type] = d

    ranked = sorted(best_by_type.values(), key=lambda d: d.impact, reverse=True)
    top = ranked[:MAX_RECOMMENDATIONS]

    score = compute_score(signals)

    return {
        "score": score.score,
        "breakdown": score.breakdown,
        "recommendations": [
            {
                "type": d.type,
                "severity": d.severity,
                "title": d.title,
                "message": d.message,
                "artifact": d.artifact,
                "status": "OPEN",
            }
        ]
    }

```

```
        for d in top
    ],
}
```

Wiring into the Celery task

```
python

# tasks/recommender.py – replaces the TODO(M5) body
from ..recommendations.engine import generate
from ..recommendations.signals import BusinessSignals, QuerySignal

@shared_task(name="answerspot_workers.tasks.recommender.generate_recommendations", auto_retry=True)
def generate_recommendations(scan_id: str):
    with session_scope() as db:
        # TODO(M5): build BusinessSignals from scan_results for this business,
        # and load review_rate/review_count from the profile source.
        signals = BusinessSignals(business_name="", category="", city="", queries=[])
        result = generate(signals, review_rate=0.0, review_count=0)
        # TODO(M5): upsert VisibilityScore snapshot + replace OPEN recommendations.
        _ = (db, result)
    return {"scan_id": scan_id}
```

Why this shape

- **Signals separate "what's true" from "what to do."** Rules never touch the DB, so they're unit-testable with hand-built `BusinessSignals` — no fixtures, no mocking a database.
- **Rules are a registry of pure functions.** New recommendation type = one function + one list entry. No touching the engine.
- **Artifacts are mostly deterministic templates.** The LLM is confined to drafting review replies, behind `_llm_review_reply`, with a working fallback — so the engine (and its tests) run without an API key, and LLM cost stays low.
- **Ranking, dedupe, and a hard cap** mean the operator always sees a short, prioritized list — the difference between a tool they act on and a wall of text they ignore.
- **Scoring is deterministic and self-documenting** — weights versioned and stored per snapshot, honoring ADR 0005 so the trend line never lies.

Test surface (M5 acceptance)

- `rule_negative_sentiment` fires CRITICAL when any query is NEGATIVE, else None.
- `rule_missing_high_intent` picks the miss with the most competitors.
- `rule_citation_gap` needs ≥ 2 gap queries to fire.
- `compute_score` is monotonic in each input and clamps to 0-100.
- `generate` never returns more than `MAX_RECOMMENDATIONS`, one per type, impact-sorted.
- Artifacts render valid JSON-LD (`faq_schema`) and non-empty copy.

0